Lecture 1. Introduction

Data science workflow, R, RStudio, tidyverse, running code, getting help

Contents

1.1 What the course is about **2**

1.1.1 Organisation and scope 2

1.2 Software **3**

1.3 Running R code **3**

1.4 Getting help **4**

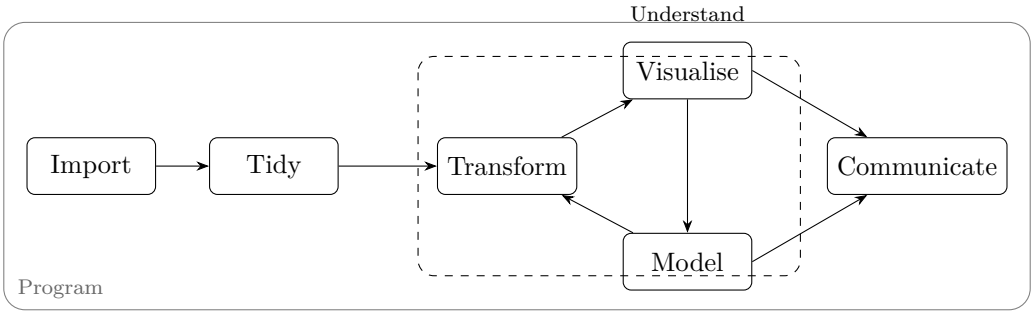
1.5 Selected exercises **4**

These notes follow the course slides, which are based on the first edition of *R for Data Science* (Wickham and Grolemund). All R code shown was run when the notes were compiled; lines starting with `##` are R's output. Material marked [Supplement] is not on the slides.

1.1 What the course is about

Motivation. Statistics courses usually start from clean data and a model. In practice most of the work happens before that: getting the data into R, cleaning it, and looking at it until the right question becomes clear. This course is about that work, done with code so that every step can be repeated and checked.

The course teaches how to collect, process, analyse and present data with a programming language. These goals are often summarised as **data science**, a term without a precise definition. A typical project passes through the following stages.



- **Import**: read data stored in a file, a database or a web API into a data frame in R.
- **Tidy**: store the data in a consistent form in which *each column is a variable and each row is an observation.*
- **Transform**: narrow down to the observations of interest (one city, the last year), create new variables from existing ones (speed from distance and time), and compute summaries (counts, means). Tidying and transforming together are called **wrangling**.
- **Visualise**: a good plot shows things you did not expect and raises new questions. Plots need a human to read them, so they do not scale well.
- **Model**: once a question is precise, a model answers it. Models are mathematical or computational tools and scale well, but every model makes assumptions that it cannot check itself.
- **Communicate**: results matter only if others can understand them.
- **Program**: programming runs through every stage. You need not be an expert programmer, but better programming lets you automate repeated tasks and solve new problems faster.

Visualising, transforming and modelling form a loop: a real analysis passes through it many times.

1.1.1 Organisation and scope

The course follows the book's order rather than the pipeline order, because the pipeline order is not the easiest way to learn the tools: visualisation and transformation first (immediate payoff), then wrangling, programming and modelling.

What is *not* covered: big data (we use small in-memory data sets; big data needs experience with small data first), languages other than R, non-rectangular data (images, sound, trees, text), and hypothesis *confirmation*. The course is about **hypothesis generation**: exploring data to find questions worth asking. Confirmatory inference is the subject of other statistics courses.

Generation versus confirmation [Supplement]

An observation can be used either to suggest a hypothesis or to test one, never both. If you search a data set for patterns and then test the pattern you found on the same data, the test is biased toward success. This is why exploration and confirmation are kept separate, a point that returns in Lecture 13.

1.2 Software

Motivation. The tools are R (the language), RStudio (the editor) and the tidyverse (a set of packages that share one design). Knowing how they fit together explains why the rest of the course looks the way it does.

R is downloaded from CRAN, the Comprehensive R Archive Network (the cloud mirror `https://cloud.r-project.org` picks a nearby server). **RStudio** is an integrated development environment for R: you type code in the console pane and press Enter to run it, or write scripts in the editor pane.

A **package** is a collection of functions, data and documentation that extends base R. The **tidyverse** is a family of packages built on a common philosophy of data and programming. Install it once, then load it in every session:

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

Loading the tidyverse attaches its core packages, which are used in almost every analysis:

package	role
`ggplot2`	graphics (Lecture 2)
`dplyr`	data transformation (Lectures 3, 6)
`tidyr`	tidying (Lecture 5)
`readr`	reading rectangular files (Lecture 5)
`tibble`	a modern data frame (Lecture 5)
`stringr`, `forcats`	strings and factors (Lectures 7, 8)
`purrr`	functional programming (Lecture 12)

R also reports conflicts: `dplyr::filter()` masks `stats::filter()`, so after loading the tidyverse `filter()` means the dplyr version. The notation `package::function()` names a function from a specific package explicitly. Updates are installed with `tidyverse_update()`.

We also use three data packages outside the tidyverse:

```
install.packages(c("nycflights13", "gapminder", "Lahman"))
```

1.3 Running R code

Motivation. Everything else in the course is written as code, so it is worth fixing how code and its output are displayed.

In the console, R prints a prompt `>`, evaluates the line you type and prints the result:

```
1 + 2
## [1] 3
```

In these notes code is shown without the prompt, and output lines start with `##` so that they can be told apart from code (and copied back without harm, since `#` starts a comment). The `[1]` marks the index of the first element printed on that line: R's basic objects are vectors.

```
seq(10, 100, by = 5)
##  [1]  10  15  20  25  30  35  40  45  50  55  60  65  70  75  80  85  90  95
## [19] 100
```

Conventions used throughout: functions are written with parentheses, `sum()`, `mean()`; other objects (variables, data sets, arguments) without, `flights`, `x`; and `dplyr::mutate()` means the function `mutate()` from the package `dplyr`.

1.4 Getting help

Inside R, `?mean` opens the documentation of `mean()`, and `??regression` searches all help pages. Outside R, searching the exact error message (often with "R" added) usually finds someone who had the same problem; Stack Overflow and blogs collected at R-bloggers are the main sources.

When asking for help, give a **reproducible example** (reprex): the packages loaded, a small data set (`dput(head(df))` prints one as R code), and the shortest code that shows the problem. Preparing a reprex often reveals the answer.

```
dput(head(mtcars[, 1:3], 3))
## structure(list(mpg = c(21, 21, 22.8), cyl = c(6, 6, 4), disp = c(160,
## 160, 108)), row.names = c("Mazda RX4", "Mazda RX4 Wag", "Datsun 710"
## ), class = "data.frame")
```

1.5 Selected exercises

Motivation. Four short problems behind the ideas of this lecture: why exploration and confirmation must use different data, why computers get $0.1 + 0.2$ "wrong", how R decides which `filter()` you mean, and what makes a table tidy.

Exercise 1.1 (exploring and confirming on the same data)

An analyst explores a data set, finds k candidate patterns, and tests each at level α. Suppose no pattern is real and the k p-values are independent and uniform on $[0, 1]$.
(a) Show that the probability that at least one test is significant is $1 - (1 - \alpha)^k$; evaluate it for $\alpha = 0.05$, $k = 20$.
(b) The analyst reports only the most striking pattern and "confirms" it with its p-value on the same data. Show that this confirmation rejects with probability $1 - (1 - \alpha)^k$, not α.
(c) Show that testing the selected pattern on new, independent data rejects with probability exactly α.

Solution [Supplement].

(a) No test is significant iff all $p_j > \alpha$. By independence this has probability $\prod_j (1 - \alpha) = (1 - \alpha)^k$, so at least one is significant with probability $1 - (1 - \alpha)^k$. For $\alpha = 0.05$, $k = 20$: $1 - 0.95^{20} = 1 - 0.358 = 0.642$.

(b) The most striking pattern has p-value $p_{(1)} = \min_j p_j$, and $p_{(1)} \le \alpha$ iff some $p_j \le \alpha$. By (a) the probability is $1 - (1-\alpha)^k$: the reported test has a false-positive rate of 64%, not 5%.

(c) Let J be the index selected from the first data set and q_J the p-value of pattern J on new data. The new data are independent of the first data and hence of J, and each q_j is uniform under the null. So

$$P(q_J \le \alpha) = \sum_j P(J = j)\, P(q_j \le \alpha \mid J = j) = \sum_j P(J = j)\, \alpha = \alpha.$$

This is the reason to keep exploration and confirmation on separate data. ■

Exercise 1.2 (why $0.1 + 0.2 \neq 0.3$)

Explain the output below. Show that 1/10 has no finite binary expansion, and characterise the decimals that are stored exactly.

```
0.1 + 0.2 == 0.3
## [1] FALSE

sprintf("%.20f", 0.1)
## [1] "0.10000000000000000555"

isTRUE(all.equal(0.1 + 0.2, 0.3))
## [1] TRUE
```

Solution [Supplement].

Step 1. R stores numbers as binary floating-point values $m/2^k$ (integers m, k, with m limited to 53 bits). A number has a finite binary expansion iff it is of this form.

Step 2. If $1/10 = m/2^k$, then $2^k = 10m$, so 5 divides 2^k, which is impossible because 2 is the only prime factor of 2^k. Hence 0.1 is stored as the nearest representable value, slightly above 0.1, as the 20 printed digits show; 0.2 and 0.3 are rounded too.

Step 3. The rounding errors do not cancel: the computed $0.1 + 0.2$ and the stored 0.3 are different doubles, so `==` returns `FALSE`. Exactly representable decimals are those $m/2^k$, such as 0.5, 0.25, 0.375.

Step 4. Comparisons of computed numbers should allow a tolerance, as `all.equal()` does (relative tolerance about 1.5×10^{-8}); in code, wrap it in `isTRUE()`, since `all.equal()` returns a description rather than `FALSE` when the values differ. ■

Exercise 1.3 (masking)

After `library(tidyverse)` the message says that `dplyr::filter()` masks `stats::filter()`. Explain the two results below.

```
stats::filter(1:5, rep(1, 3) / 3)
## Time Series:
## Start = 1
## End = 5
## Frequency = 1
## [1] NA  2  3  4 NA

filter(1:5, rep(1, 3) / 3)
## Error in UseMethod("filter"): no applicable method for 'filter' applied to an object of
    class "c('integer', 'numeric')"
```

Solution [Supplement].

Step 1. `stats::filter()` applies a linear filter to a series; with weights $(1/3, 1/3, 1/3)$ it is a centred moving average, $(1+2+3)/3 = 2$, $(2+3+4)/3 = 3$, $(3+4+5)/3 = 4$, with `NA` at the two ends where the window does not fit.

Step 2. An unqualified name is looked up along the search path, `search()`, from the global environment through the attached packages to base R. `library()` attaches a package near the front of this list, ahead of `stats`, so `filter` now finds dplyr's function first.

Step 3. `dplyr::filter()` is a generic for data frames and has no method for a numeric vector, hence the error. Writing `package::function()` bypasses the search path, which is how both functions remain usable. ■

Exercise 1.4 (what is tidy; adapted from R4DS, Exercise 12.2.1.1)

A table has one row per country and columns `country`, `1999` and `2000`, the last two holding the number of tuberculosis cases in that year. Name the variables and the observations, explain why the table is not tidy, and describe the tidy table for c countries.

Solution [Supplement].

Step 1. The variables are country, year and cases; an observation is one country in one year. Here the values of the variable year (1999, 2000) are used as column names, and the variable cases is spread over two columns, so a column is not a variable and a row holds two observations.

Step 2. The tidy table has the three columns `country`, `year`, `cases` and one row per country–year pair, i.e. $2c$ rows; each of its $2c$ values of `cases` is one of the $2c$ entries of the original year columns, so no information is lost. This reshaping is what `gather()` and `pivot_longer()` do (Lecture 5). ■